UD4. Concorrencia e bloqueo de datos

Nesta parte da unidade estudarase como xestiona MariaDB o acceso simultáneo de varios usuarios á mesma base de datos. Analizaranse os problemas que poden producirse cando varias transaccións consultan ou modifican os mesmos rexistros e os mecanismos que ofrece InnoDB para manter a integridade e a consistencia dos datos.

1. Introdución á concorrencia

1.1. Que é a concorrencia?

Nun sistema de bases de datos é habitual que varios usuarios ou aplicacións accedan simultaneamente á mesma información. Por exemplo, nunha aplicación bancaria, dúas persoas poden consultar unha conta mentres outra aplicación está realizando unha transferencia.

Denomínase **concorrencia** á execución simultánea ou intercalada de operacións realizadas por diferentes conexións á base de datos.

MariaDB permite que varias conexións traballen ao mesmo tempo. Non obstante, cando varias transaccións acceden aos mesmos rexistros, poden aparecer conflitos se non se controla correctamente a orde das operacións.

> **Exemplo: modificación simultánea dunha conta**
>
> Unha conta bancaria ten un saldo de 1.000 euros. Dúas operacións independentes intentan retirar 100 e 200 euros, respectivamente.
>
> - O usuario A consulta o saldo: 1.000 euros.
> - O usuario B consulta o saldo: 1.000 euros.
> - O usuario A calcula o novo saldo: 900 euros.
> - O usuario B calcula o novo saldo: 800 euros.
> - O usuario A actualiza o saldo a 900 euros.
> - O usuario B actualiza o saldo a 800 euros.
>
> O resultado final é 800 euros, pero deberían quedar 700 euros. Unha das actualizacións perdeu o efecto da outra.

1.2. Problemas habituais da concorrencia

Se non se utilizan mecanismos adecuados de illamento e bloqueo, poden producirse diferentes anomalías.

Actualización perdida (Lost Update)

Prodúcese cando dúas transaccións len o mesmo valor, calculan modificacións independentes e unha delas sobrescribe o resultado da outra.

Orde	Transacción A	Transacción B	Saldo
1	Le o saldo		1.000 €
2		Le o saldo	1.000 €
3	Calcula 900 €		1.000 €
4		Calcula 800 €	1.000 €
5	Actualiza a 900 €		900 €
6		Actualiza a 800 €	800 €

A actualización da transacción A queda perdida porque a transacción B escribe un valor calculado a partir dunha lectura anterior.

Lectura non confirmada (Dirty Read)

Prodúcese cando unha transacción consulta cambios realizados por outra transacción que aínda non executou `COMMIT`.

Situación:

1. A transacción A modifica o saldo dunha conta de 1.000 a 500 euros.
2. A transacción B consulta o saldo e obtén 500 euros.
3. A transacción A executa `ROLLBACK`.
4. O saldo real volve ser 1.000 euros.

A transacción B utilizou un valor que nunca chegou a confirmarse. Esta situación denomínase lectura non confirmada.

Lectura non repetible (Non-repeatable Read)

Prodúcese cando unha transacción consulta un rexistro, outra transacción modifica e confirma ese rexistro e, ao repetirse a consulta inicial, obtense un valor diferente.

> 1. A transacción A consulta o saldo: 1.000 euros.
> 2. A transacción B cambia o saldo a 800 euros e executa `COMMIT`.
> 3. A transacción A volve consultar o saldo e obtén 800 euros.
>
> A mesma consulta devolveu valores diferentes dentro da mesma transacción.

Lectura fantasma (Phantom Read)

Prodúcese cando unha transacción executa unha consulta que devolve un conxunto de rexistros e, posteriormente, outra transacción insire ou elimina rexistros que cumpren esa mesma condición.

> A transacción A consulta todos os alumnos matriculados no curso DAW1 e obtén 20 resultados.
>
> A transacción B insire un novo alumno matriculado en DAW1 e confirma a operación.
>
> Se A repite a consulta e obtén 21 resultados, aparece un rexistro novo que antes non estaba no resultado.

> **Importante:** a lectura non repetible afecta ao valor dun rexistro que xa existía. A lectura fantasma afecta ao conxunto de rexistros que devolve unha consulta.

2. Concepto de bloqueo

2.1. Que é un bloqueo?

Un **bloqueo (lock)** é un mecanismo que utiliza o sistema xestor de bases de datos para controlar o acceso simultáneo aos datos. Permite impedir que unha transacción interfira de maneira incorrecta nas operacións doutra.

Cando unha transacción bloquea un rexistro, outras transaccións poden ter que esperar para acceder a el, dependendo do tipo de bloqueo solicitado e da operación que pretendan realizar.

2.2. Bloqueos compartidos e exclusivos

Os dous tipos fundamentais de bloqueo son os compartidos e os exclusivos.

Bloqueo compartido (Shared Lock)

Permite que unha transacción lea un rexistro impedindo que outra transacción o modifique mentres o bloqueo estea activo.

- Varias transaccións poden manter bloqueos compartidos sobre o mesmo rexistro.
- Permite realizar lecturas simultáneas.
- Non é compatible cun bloqueo exclusivo sobre ese mesmo rexistro.

Bloqueo exclusivo (Exclusive Lock)

Permite que unha transacción modifique un rexistro e impide que outras transaccións obteñan bloqueos incompatibles sobre el.

- Utilízase habitualmente nas operacións de modificación e eliminación.
- Unha segunda transacción que necesite modificar o mesmo rexistro terá que esperar.
- Un bloqueo exclusivo tamén impide obter un bloqueo compartido incompatible.

Bloqueo solicitado	Compartido existente	Exclusivo existente
Compartido	Compatible	Debe esperar
Exclusivo	Debe esperar	Debe esperar

2.3. Bloqueo de táboas e bloqueo de rexistros

Os bloqueos poden afectar a diferentes unidades de información.

Bloqueo de táboa

Afecta a unha táboa completa. Mentres está activo, pode limitar ou impedir operacións doutras conexións sobre esa táboa.

MariaDB permite solicitar bloqueos explícitos mediante `LOCK TABLES`.

```
LOCK TABLES alumnos READ;

-- Consultas sobre a táboa
SELECT * FROM alumnos;

UNLOCK TABLES;
```

Tamén se pode solicitar un bloqueo de escritura:

```
LOCK TABLES alumnos WRITE;

UPDATE alumnos
SET edad = edad + 1
WHERE id = 1;

UNLOCK TABLES;
```

Bloqueo de rexistro

Afecta a rexistros concretos dunha táboa. É o mecanismo habitual empregado por InnoDB para permitir que diferentes transaccións traballen simultaneamente sobre rexistros distintos.

> Se unha transacción bloquea o alumno co identificador 1, outra transacción pode traballar co alumno co identificador 2 sen necesidade de esperar, sempre que non existan outros conflitos.

InnoDB tamén pode bloquear intervalos de índices ou rexistros próximos (bloqueos de rango e de intervalo) para garantir a consistencia das consultas, especialmente nalgúns niveis de illamento.

> **Nota:** os bloqueos de táboa explícitos con `LOCK TABLES` teñen regras propias e poden provocar unha confirmación implícita da transacción activa. Non deben confundirse cos bloqueos internos que InnoDB utiliza para xestionar as transaccións.

3. Niveis de illamento das transaccións

3.1. Concepto de illamento

O nivel de illamento determina ata que punto unha transacción pode observar os cambios realizados por outras transaccións que se están executando simultaneamente.

Un nivel de illamento máis estrito ofrece maiores garantías de consistencia, pero pode reducir a concorrencia e incrementar as esperas entre transaccións.

MariaDB, mediante InnoDB, admite catro niveis de illamento:

3.2. READ UNCOMMITTED

É o nivel de illamento menos estrito. Permite que unha transacción consulte cambios doutras transaccións que aínda non foron confirmados.

- Pode producir lecturas non confirmadas.
- Pode observar valores que posteriormente serán desfeitos mediante `ROLLBACK`.
- Ofrece pouca protección fronte ás anomalías de concorrencia.

```
SET SESSION TRANSACTION ISOLATION LEVEL
READ UNCOMMITTED;

START TRANSACTION;

SELECT * FROM contas
WHERE id = 1;

COMMIT;
```

3.3. READ COMMITTED

Cada consulta só pode observar os cambios confirmados antes do inicio desa consulta. Non permite lecturas de datos pendentes doutras transaccións.

- Evita as lecturas non confirmadas.
- Pode producir lecturas non repetibles.
- Pode producir lecturas fantasma.
- Unha mesma transacción pode obter resultados diferentes ao repetir unha consulta.

```
SET SESSION TRANSACTION ISOLATION LEVEL
READ COMMITTED;

START TRANSACTION;

SELECT * FROM contas
WHERE id = 1;

COMMIT;
```

3.4. REPEATABLE READ

Garante que as lecturas consistentes realizadas dentro dunha transacción manteñan a mesma visión dos datos confirmados ao comezo da transacción.

- Evita as lecturas non confirmadas.
- Evita as lecturas non repetibles nas lecturas consistentes habituais de InnoDB.
- InnoDB utiliza mecanismos de control de versións e bloqueos de rango para manter a consistencia.
- É o nivel de illamento predeterminado habitual de InnoDB en MariaDB.

```
SET SESSION TRANSACTION ISOLATION LEVEL
REPEATABLE READ;

START TRANSACTION;

SELECT * FROM contas
WHERE id = 1;

COMMIT;
```

Cómpre distinguir as lecturas consistentes normais das lecturas bloqueantes, como `SELECT ... FOR UPDATE`. Estas últimas utilizan regras de bloqueo e non se comportan exactamente igual que unha consulta `SELECT` normal.

3.5. SERIALIZABLE

É o nivel de illamento máis estrito. Busca que o resultado das transaccións concorrentes sexa equivalente a unha execución secuencial, unha tras outra.

- Ofrece un maior control sobre as anomalías de concorrencia.
- Incrementa a utilización de bloqueos.
- Pode provocar máis esperas e conflitos entre transaccións.

- Pode reducir o rendemento cando existen moitos accesos simultáneos.

```
SET SESSION TRANSACTION ISOLATION LEVEL
SERIALIZABLE;

START TRANSACTION;

SELECT * FROM contas
WHERE id = 1;

COMMIT;
```

3.6. Comparación dos niveis de illamento

Nivel	Lectura non confirmada	Lectura non repetible	Lectura fantasma
READ UNCOMMITTED	Si	Si	Si
READ COMMITTED	Non	Si	Si
REPEATABLE READ	Non	Non	Non nas lecturas consistentes habituais de InnoDB
SERIALIZABLE	Non	Non	Non

Esta táboa resume o comportamento habitual das lecturas en InnoDB. A implementación concreta depende do tipo de consulta, dos índices dispoñibles e dos bloqueos utilizados.

3.7. Consultar e modificar o nivel de illamento

Pódese consultar o nivel de illamento da conexión actual:

```
SELECT @@SESSION.tx_isolation;
```

Tamén se pode consultar o nivel global:

```
SELECT @@GLOBAL.tx_isolation;
```

Para cambiar o nivel da conexión actual:

```
SET SESSION TRANSACTION ISOLATION LEVEL
READ COMMITTED;
```

Para cambiar o nivel global, que se aplicará ás novas conexións:

```
SET GLOBAL TRANSACTION ISOLATION LEVEL
READ COMMITTED;
```

> O cambio de nivel de illamento dunha sesión debe realizarse antes de iniciar a transacción que se quere executar con ese nivel. O cambio global require os permisos correspondentes e non modifica automaticamente o nivel das conexións xa abertas.

4. Lectura e modificación de rexistros con bloqueo

4.1. Por que bloquear os rexistros antes de modificalos?

En determinadas operacións non é suficiente con consultar un valor e, posteriormente, modificalo. Entre ambas as operacións outra transacción podería modificar o mesmo rexistro.

Para evitar este problema, InnoDB permite realizar consultas que bloquean os rexistros seleccionados. Deste xeito, a transacción pode consultar os datos e realizar as modificacións mantendo o control sobre eles.

4.2. SELECT ... FOR UPDATE

A cláusula `FOR UPDATE` realiza unha lectura bloqueante. Os rexistros seleccionados quedan bloqueados mediante bloqueos exclusivos para que outras transaccións non poidan modificalos ata que a transacción actual remate.

```
START TRANSACTION;

SELECT *
FROM contas
WHERE id = 1
FOR UPDATE;

UPDATE contas
SET saldo = saldo - 100
WHERE id = 1;

COMMIT;
```

O bloqueo mantense ata que se executa `COMMIT` ou `ROLLBACK`.

Se outra conexión intenta modificar o mesmo rexistro, normalmente terá que esperar a que remate a transacción que mantén o bloqueo.

> **Importante:** en InnoDB, `FOR UPDATE` debe utilizarse dentro dunha transacción activa. Se se executa cunha transacción independente por cada sentenza (por exemplo, co modo `autocommit` activado e sen iniciar unha transacción explícita), o bloqueo non se mantén para unha operación posterior.

4.3. LOCK IN SHARE MODE

A cláusula `LOCK IN SHARE MODE` permite realizar unha lectura mantendo un bloqueo compartido sobre os rexistros seleccionados.

```
START TRANSACTION;

SELECT *
FROM contas
WHERE id = 1
LOCK IN SHARE MODE;

COMMIT;
```

Mentres a transacción mantén o bloqueo compartido:

- Outras transaccións poden realizar lecturas compatibles.
- Outras transaccións non poden modificar os rexistros bloqueados mediante un bloqueo incompatible.
- Unha operación que necesite un bloqueo exclusivo terá que esperar.

4.4. Comparación das dúas cláusulas

Característica	FOR UPDATE	LOCK IN SHARE MODE
Tipo de bloqueo	Exclusivo	Compartido
Permite outras lecturas compatibles	Non con bloqueo incompatible	Si
Impide modificacións concorrentes	Si	Si, mentres se manteña o bloqueo
Uso habitual	Ler e modificar datos	Ler datos que non deben cambiar durante a operación

4.5. Exemplo: transferencia bancaria segura

Supoñamos que se necesita transferir 100 euros da conta 1 á conta 2. É necesario comprobar os saldos e evitar que outra transacción modifique as contas durante a operación.

```
START TRANSACTION;

-- Bloqueamos as dúas contas.
SELECT id, saldo
FROM contas
WHERE id IN (1, 2)
ORDER BY id
FOR UPDATE;

-- Comprobamos que a conta de orixe ten saldo suficiente.
-- Os identificadores 1 e 2 deben existir previamente.

UPDATE contas
SET saldo = saldo - 100
WHERE id = 1
  AND saldo >= 100;

-- Comprobamos que se modificou exactamente unha fila.
SELECT ROW_COUNT();

UPDATE contas
SET saldo = saldo + 100
WHERE id = 2;

-- Comprobamos tamén esta segunda modificación.
SELECT ROW_COUNT();

COMMIT;
```

> Este código mostra a secuencia de operacións, pero non debe considerarse un procedemento completo de produción: antes de executar `COMMIT` hai que verificar que existen as dúas contas, que a primeira actualización afectou exactamente unha fila e que a segunda tamén afectou exactamente unha fila. Se algunha comprobación falla, débese executar `ROLLBACK`.

Unha implementación real tamén debe contemplar que as dúas contas poden ser iguais, que os saldos deben cumprir as regras de negocio e que os erros durante a operación deben provocar a súa cancelación.

5. Deadlocks e problemas de bloqueo

5.1. Que é un deadlock?

Un **deadlock** ou bloqueo mutuo prodúcese cando dúas ou máis transaccións quedan esperando por recursos que están bloqueados entre si.

Ningunha das transaccións pode continuar porque cada unha necesita que a outra libere un bloqueo.

> **Exemplo de bloqueo mutuo**
>
> Temos dúas transaccións que modifican dúas contas:
>
> 1. A transacción A bloquea a conta 1.
> 2. A transacción B bloquea a conta 2.
> 3. A transacción A intenta bloquear a conta 2 e queda esperando.
> 4. A transacción B intenta bloquear a conta 1 e queda esperando.
>
> A transacción A espera que B libere a conta 2, mentres que B espera que A libere a conta 1.

5.2. Simulación dun deadlock

Para reproducir a situación anterior necesitamos dúas conexións independentes a MariaDB. En MySQL Workbench pódense abrir dúas pestanas de conexión á mesma base de datos.

Preparación:

```
CREATE DATABASE IF NOT EXISTS concorrencia;

USE concorrencia;

CREATE TABLE contas (
    id INT PRIMARY KEY,
    saldo DECIMAL(10,2) NOT NULL
) ENGINE=InnoDB;

INSERT INTO contas (id, saldo)
VALUES
(1, 1000.00),
(2, 2000.00);
```

Executa primeiro as seguintes instrucións na conexión A:

```
START TRANSACTION;

UPDATE contas
SET saldo = saldo + 100
WHERE id = 1;
```

A continuación, executa na conexión B:

```
START TRANSACTION;

UPDATE contas
SET saldo = saldo + 200
WHERE id = 2;
```

Agora a conexión A ten bloqueada a conta 1 e a conexión B ten bloqueada a conta 2.

Na conexión A executa:

```
UPDATE contas
SET saldo = saldo + 50
WHERE id = 2;
```

A operación queda esperando porque a conta 2 está bloqueada pola conexión B.

Sen cancelar a operación anterior, na conexión B executa:

```
UPDATE contas
SET saldo = saldo + 75
WHERE id = 1;
```

Agora B necesita a conta 1, que está bloqueada por A. MariaDB detectará o deadlock e normalmente cancelará unha das transaccións para romper o bloqueo mutuo.

A transacción seleccionada como vítima recibirá un erro de bloqueo mutuo. A outra poderá continuar cando se liberen os recursos necesarios.

> **Importante:** InnoDB detecta automaticamente os deadlocks e selecciona unha transacción para desfacela. A aplicación debe estar preparada para repetir a operación cancelada cando resulte apropiado.

5.3. Causas habituais dos deadlocks

- Varias transaccións modifican os mesmos rexistros en diferente orde.
- As transaccións manteñen bloqueos durante demasiado tempo.
- As operacións modifican moitos rexistros simultaneamente.

- As consultas non utilizan índices adecuados e bloquean máis rexistros dos necesarios.
- As aplicacións realizan operacións de lectura e escritura en diferentes ordes.

5.4. Medidas para evitar ou reducir os deadlocks

Non sempre é posible evitar completamente os deadlocks, pero pódese reducir a súa frecuencia aplicando boas prácticas:

1. **Manter unha orde constante de acceso aos datos.** Se varias transaccións modifican as contas 1 e 2, deben bloquealas sempre na mesma orde.
2. **Reducir a duración das transaccións.** Non manter transaccións abertas mentres se realizan operacións externas ou se espera unha intervención do usuario.
3. **Utilizar índices adecuados.** As consultas deben localizar os rexistros necesarios sen examinar ou bloquear unha cantidade excesiva de datos.
4. **Modificar só os rexistros necesarios.** Evitar operacións que afecten a máis filas das requiridas.
5. **Controlar os erros.** Se InnoDB cancela unha transacción por deadlock, a aplicación debe xestionar o erro e, se procede, repetir a operación.

5.5. Consultar información sobre os bloqueos

MariaDB ofrece diferentes mecanismos para analizar o estado das transaccións e detectar problemas.

Consultar as conexións e operacións activas:

```
SHOW PROCESSLIST;
```

Consultar información do motor InnoDB:

```
SHOW ENGINE INNODB STATUS;
```

Esta última instrución permite consultar información sobre transaccións, bloqueos e o último deadlock detectado.

Tamén se pode activar o rexistro de todos os deadlocks no rexistro de erros mediante a variable correspondente:

```
SET GLOBAL innodb_print_all_deadlocks = ON;
```

Esta configuración require permisos administrativos. O rexistro dependerá da configuración de logs do servidor MariaDB.

6. Práctica de concorrencia con MySQL Workbench

6.1. Preparación do entorno

Para comprobar o funcionamento dos bloqueos é necesario utilizar dúas conexións independentes á mesma base de datos MariaDB. Cada conexión representará un usuario diferente traballando simultaneamente.

Abre MySQL Workbench e establece dúas conexións ao mesmo servidor. Podes utilizar dúas pestanas SQL abertas sobre conexións distintas.

> **Importante:** dúas pestanas que utilizan a mesma conexión non permiten simular correctamente dúas transaccións independentes. Debes comprobar que cada pestana utiliza unha conexión diferente.

6.2. Crear a base de datos de probas

Executa unha soa vez o seguinte código desde calquera das conexións:

```
CREATE DATABASE IF NOT EXISTS concorrencia;

USE concorrencia;

CREATE TABLE produtos (
    id INT PRIMARY KEY,
    nome VARCHAR(50) NOT NULL,
    stock INT NOT NULL
) ENGINE=InnoDB;

INSERT INTO produtos (id, nome, stock)
VALUES
(1, 'Teclado', 20),
(2, 'Rato', 30),
(3, 'Monitor', 10);

SELECT * FROM produtos;
```

A táboa representa un pequeno inventario. As dúas conexións traballarán sobre os mesmos produtos.

6.3. Proba 1: actualización bloqueada

O obxectivo é comprobar que unha transacción pode impedir que outra modifique o mesmo rexistro.

Conexión A:

```
USE concorrencia;

START TRANSACTION;

UPDATE produtos
SET stock = stock - 2
WHERE id = 1;
```

A conexión A modifica o stock do teclado, pero aínda non confirma os cambios. O rexistro permanece bloqueado.

Conexión B:

```
USE concorrencia;

UPDATE produtos
SET stock = stock - 3
WHERE id = 1;
```

A operación da conexión B queda esperando porque a conexión A mantén un bloqueo sobre ese rexistro.

Volvemos á conexión A:

```
COMMIT;
```

Ao confirmar a transacción, libérase o bloqueo. A conexión B pode continuar coa súa actualización.

Comproba o resultado:

```
SELECT * FROM produtos
WHERE id = 1;
```

O stock final debe ser 15 unidades: 20 iniciais, menos 2 da conexión A e menos 3 da conexión B.

6.4. Proba 2: cancelar unha transacción

Nesta proba comprobarase que un cambio pendente pode desfacerse e que, ao facelo, se libera o bloqueo.

Conexión A:

```
START TRANSACTION;

UPDATE produtos
SET stock = stock - 5
WHERE id = 2;
```

Conexión B:

```
UPDATE produtos
SET stock = stock - 1
WHERE id = 2;
```

A conexión B quedará esperando. Agora, na conexión A, executa:

```
ROLLBACK;
```

O cambio da conexión A desfárase e libérase o bloqueo. A conexión B poderá realizar a súa modificación.

Consulta o resultado:

```
SELECT * FROM produtos
WHERE id = 2;
```

O stock final será 29 unidades, xa que a operación da conexión A foi cancelada e só se aplicou a da conexión B.

6.5. Proba 3: bloqueo de lectura con FOR UPDATE

Nesta proba observarase como unha consulta tamén pode bloquear rexistros para preparar unha modificación posterior.

Conexión A:

```
START TRANSACTION;

SELECT *
FROM produtos
WHERE id = 3
FOR UPDATE;
```

A consulta obtén un bloqueo exclusivo sobre o rexistro seleccionado.

Conexión B:

```
UPDATE produtos
SET stock = stock - 1
WHERE id = 3;
```

A actualización queda esperando ata que a conexión A libere o bloqueo.

Conexión A:

```
COMMIT;
```

A conexión B poderá continuar coa súa operación.

6.6. Proba 4: comparación dos niveis de illamento

Nesta proba analizarase a diferenza entre `READ COMMITTED` e `REPEATABLE READ`.

Primeiro, comproba que o produto 1 ten un stock coñecido:

```
SELECT * FROM produtos
WHERE id = 1;
```

Conexión A:

```
SET SESSION TRANSACTION ISOLATION LEVEL
READ COMMITTED;

START TRANSACTION;

SELECT stock
FROM produtos
WHERE id = 1;
```

Conexión B:

```
UPDATE produtos
SET stock = stock + 10
WHERE id = 1;

COMMIT;
```

De novo na conexión A:

```
SELECT stock
FROM produtos
WHERE id = 1;
```

En `READ COMMITTED`, a segunda consulta pode observar o novo stock confirmado pola conexión B.

Agora repite a proba cambiando o nivel de illamento da conexión A a `REPEATABLE READ`:

```
SET SESSION TRANSACTION ISOLATION LEVEL
REPEATABLE READ;

START TRANSACTION;

SELECT stock
FROM produtos
WHERE id = 1;
```

Desde a conexión B, modifica de novo o stock e confirma:

```
UPDATE produtos
SET stock = stock + 5
WHERE id = 1;

COMMIT;
```

Volve consultar o stock desde a conexión A:

```
SELECT stock
FROM produtos
WHERE id = 1;
```

A lectura consistente da conexión A manterá a visión dos datos correspondente ao inicio da súa transacción, polo que non observará a modificación confirmada pola conexión B.

Remata a transacción da conexión A:

```
COMMIT;
```

> Para comparar correctamente os resultados, cada proba debe realizarse en dúas conexións independentes. Non se debe executar o segundo `SELECT` despois de confirmar a transacción A, xa que iso iniciaría unha nova transacción e unha nova visión dos datos.

6.7. Proba 5: reprodución dun deadlock

Utiliza as dúas conexións para reproducir o bloqueo mutuo descrito no apartado 5.2, utilizando os rexistros 1 e 2 da táboa `produtos`.

A secuencia debe ser:

1. Iniciar unha transacción na conexión A e actualizar o produto 1.
2. Iniciar unha transacción na conexión B e actualizar o produto 2.
3. Desde A, intentar actualizar o produto 2.
4. Desde B, intentar actualizar o produto 1.
5. Observar a resposta de MariaDB e identificar a transacción cancelada.
6. Comprobar o estado final dos rexistros.

Ao rematar, consulta:

```
SHOW ENGINE INNODB STATUS;
```

Localiza a información correspondente ao último deadlock detectado e identifica as transaccións e os bloqueos implicados.

7. Resumo dos conceptos fundamentais

Concepto	Descrición
Concorrencia	Acceso simultáneo de varias transaccións á base de datos.
Actualización perdida	Unha modificación sobrescribe o resultado doutra.
Lectura non confirmada	Unha transacción le cambios que outra aínda non confirmou.
Lectura non repetible	Unha consulta sobre o mesmo rexistro devolve valores diferentes.
Lectura fantasma	Unha consulta devolve un conxunto diferente de rexistros.
Bloqueo compartido	Permite lecturas compatibles e impide modificacións incompatibles.
Bloqueo exclusivo	Reserva os rexistros para unha operación de modificación.
FOR UPDATE	Realiza unha lectura con bloqueo exclusivo.
LOCK IN SHARE MODE	Realiza unha lectura con bloqueo compartido.
Deadlock	Varias transaccións esperan por recursos bloqueados entre si.
REPEATABLE READ	Mantén unha visión consistente dos datos nas lecturas habituais da transacción.

Boas prácticas no control da concorrencia

- Utilizar InnoDB nas táboas que necesiten transaccións e bloqueo de rexistros.
- Escoller un nivel de illamento adecuado para cada aplicación.
- Utilizar `SELECT ... FOR UPDATE` cando sexa necesario ler e modificar os mesmos rexistros de forma segura.

- Manter as transaccións abertas durante o menor tempo posible.
- Acceder aos rexistros nunha orde constante para reducir os deadlocks.
- Crear índices adecuados para mellorar a eficiencia das consultas.
- Controlar os erros e executar `ROLLBACK` cando unha operación non poida completarse correctamente.
- Comprobar os datos afectados antes de confirmar unha transacción.